Super Market Billing System

Automated Retail Transaction & Inventory Management

Chandra Prakash Choudhary | Data Analyst

Table of Contents

1. Executive Summary
2. Project Overview
3. Inventory & Discounts
4. System Workflow
5. Data Analyst Insights
6. Code Logic Breakdown
7. Full Source Code
8. Program Output
9. Conclusion

Executive Summary & Problem Statement

Abstract

Problem Statement

The **Star Daily Mart Billing System** is a Python-based console application designed to streamline the retail checkout process. By automating calculations, applying conditional logic for discounts, and generating professional itemized receipts, this system eliminates manual errors and enhances customer satisfaction.

Traditional manual billing is slow and error-prone, especially when handling complex volume-based discounts (e.g., "Buy 5kg, get 5% off"). This project solves these issues by providing:

- ✅ Automated arithmetic operations
- ✅ Dynamic inventory lookup
- ✅ Real-time discount application

Project Overview

This Point of Sale (POS) simulation handles the end-to-end flow of a supermarket transaction. From capturing customer details (Name, Phone) to building a shopping cart and printing a final tax invoice, the system is designed for reliability and ease of use.

Data Structure
Utilizes Python `dictionaries` for O(1) inventory lookup and `lists of tuples` for efficient cart management.

Logic Core
Implements complex `if-elif-else` structures to evaluate promotional eligibility dynamically during the billing phase.

Inventory & Promotional Structure

Available Items

ID	Product Name	Price (INR)	Unit
1	Rice	50	kg
2	Wheat Flour	45	kg
3	Sugar	40	kg

ID	Product Name	Price (INR)	Unit
4	Cooking Oil	140	litre
5	Milk	60	litre
6	Bread	30	packet
7	Eggs	6	piece
8	Soap	35	piece
9	Shampoo	120	bottle
10	Biscuits	25	packet

Active Discount Campaigns

TOP SAVER

Rice Bonanza

Buy **5 kg** or more

5% OFF

Applied on total Rice value

DAILY NEEDS

Milk Splash

Buy **3 Litres** or more

10% OFF

Applied on total Milk value

SNACKS

Biscuit Crunch

Buy **2 Packets** or more

15% OFF

Applied on total Biscuit value

System Workflow

1. **User Authentication**
   System prompts for Customer Name and validates a 10-digit Phone Number.

2. **Menu Display**
   Shows catalogue of products with IDs, Names, and Unit Prices.

3. **Cart Building Loop**
   User selects Item IDs and Quantities. Validates inputs (integers/floats) and adds to the temporary cart list.

4. **Processing & Billing**
   Iterates through the cart, calculates line totals, checks discount conditions, and computes Grand Total.

Data Analyst Perspective

Discount Optimization

The code implements **Conditional Logic Thresholds** (e.g., Rice > 5kg). By analyzing transaction logs (output), we can determine if these thresholds are effectively driving bulk purchases.

Market Basket Analysis

The `cart` list structure captures item-sets. This data is foundational for **Association Rule Mining** (e.g., "Do people who buy Milk also buy Biscuits?").

Enhancement Insight: Currently, the system stores data in volatile memory (RAM). Integrating a CSV logger or SQL database would enable longitudinal time-series analysis to predict stock depletion rates.

Code Logic Breakdown

1. Data Dictionary Setup - The `items` dictionary acts as a NoSQL-style database, mapping IDs to tuples of attributes.

```
items = {
    1: ("Rice", 50, "kg"),
    2: ("Wheat Flour", 45, "kg"),
    # ... other items
}
```

2. Robust Input Validation - The `input_int` function uses `try-except` blocks to handle errors.

```
def input_int(prompt):
    while True:
        try:
            return int(input(prompt).strip())
        except ValueError:
            print("Please enter a valid integer.")
```

Complete Source Code

Below is the complete, executable Python script for the Super Market Billing System.

```python
# Super Market Billing System
# This program allows users to purchase multiple items,
# applies discounts and generates a final bill.

# Dictionary storing item details: item_no -> (name, price, unit)
items = {
    1: ("Rice", 50, "kg"),
    2: ("Wheat Flour", 45, "kg"),
    3: ("Sugar", 40, "kg"),
    4: ("Cooking Oil", 140, "litre"),
    5: ("Milk", 60, "litre"),
    6: ("Bread", 30, "packet"),
    7: ("Eggs", 6, "piece"),
    8: ("Soap", 35, "piece"),
    9: ("Shampoo", 120, "bottle"),
    10: ("Biscuits", 25, "packet"),
}

# Takes customer name and phone number with validation
def user_input():
    user_name = input("Enter your name: ")
    user_phone = input("Enter your phone number: ")

    # Validate phone number
    if not user_phone.isdigit() or len(user_phone) != 10:
        print("Invalid phone number. Please enter a 10-digit number.")
        return user_input()

    return user_name, user_phone

# Displays store header
def show_header():
    print("=" * 50)
    print("WELCOME TO STAR DAILY MART".center(50))
    print("=" * 50)
    print("Fresh  Affordable  Trusted\n")

# Displays available items
def show_menu():
    print("-" * 50)
    print("AVAILABLE ITEMS (price per unit)")
    print("-" * 50)
    for no, (name, price, unit) in items.items():
        print(f"{no:2d}. {name:<14} : {price} per {unit}")
    print("-" * 50)

# Takes integer input safely
def input_int(prompt):
    while True:
        try:
            return int(input(prompt).strip())
        except ValueError:
            print("Please enter a valid integer.")

# Takes float input safely
```

```python
def input_float(prompt):
    while True:
        try:
            val = float(input(prompt).strip())
            if val <= 0:
                print("Please enter a positive number.")
                continue
            return val
        except ValueError:
            print("Please enter a valid number.")

# Displays discount offers
def discount_offers():
    print("\nToday's Discount Offers:")
    print("1. Buy 5 kg or more of Rice and get 5% off on Rice.")
    print("2. Buy 3 litres or more of Milk and get 10% off on Milk.")
    print("3. Buy 2 packets or more of Biscuits and get 15% off on Biscuits.\

# Main function controlling the billing process
def main():
    # Get customer details
    user_name, user_phone = user_input()

    show_header()
    show_menu()
    discount_offers()

    cart = []  # Stores purchased items

    # Loop to add multiple items
    while True:
        item_no = input_int("Enter item number to buy: ")
        if item_no not in items:
            print("Invalid item number. Try again.")
            continue

        name, price, unit = items[item_no]
        qty = input_float(f"Enter quantity of {name} (in {unit}): ")

        # Store item details in cart
        cart.append((item_no, name, price, unit, qty))
        print(f"Added {qty} {unit} of {name} at {price} each.\n")

        more = input("Do you want to add more items? (y/n): ").strip().lower(
        if more not in ("y", "yes"):
            break

    # Print final bill
    print("\n" + "=" * 20 + " FINAL BILL " + "=" * 20)

    subtotal = 0.0     # Total before discount
    grand_total = 0.0  # Total after discount

    # Process each item in cart
    for item_no, name, price, unit, qty in cart:
        original_line = price * qty
        subtotal += original_line
```

```
        discount = 0

        # Apply item-specific discounts
        if item_no == 1 and qty >= 5:
            discount = 0.05
            print("5% discount applied on Rice.")
        elif item_no == 5 and qty >= 3:
            discount = 0.10
            print("10% discount applied on Milk.")
        elif item_no == 10 and qty >= 2:
            discount = 0.15
            print("15% discount applied on Biscuits.")

        # Calculate final price after discount
        if discount > 0:
            print(f"Original price: {original_line:.2f}")
            final_line = original_line * (1 - discount)
        else:
            final_line = original_line

        print(f"{name:14}  {qty:6.2f} {unit:<6} x {price:6.2f}  =  {final_lir
        grand_total += final_line

    # Display totals and customer info
    print("-" * 56)
    print(f"Subtotal before discount: {subtotal:.2f}")
    print(f"GRAND TOTAL: {grand_total:7.2f}")
    print("=" * 56)
    print("Thank you for shopping at STAR DAILY MART!".center(56))
    print("Visit Again!".center(56))
    print(f"Customer Name : {user_name}")
    print(f"Phone Number  : {user_phone}")

# Program execution starts here
if __name__ == "__main__":
    main()
```

Actual Program Execution Screenshot

```
Enter your name: XYZ
Enter your phone number: 1234567890
=====================================================
            WELCOME TO STAR DAILY MART
=====================================================
Fresh • Affordable • Trusted

-----------------------------------------------------
AVAILABLE ITEMS (price per unit)
-----------------------------------------------------
 1. Rice            : ₹50 per kg
 2. Wheat Flour     : ₹45 per kg
 3. Sugar           : ₹40 per kg
 4. Cooking Oil     : ₹140 per litre
 5. Milk            : ₹60 per litre
 6. Bread           : ₹30 per packet
 7. Eggs            : ₹6 per piece
 8. Soap            : ₹35 per piece
 9. Shampoo         : ₹120 per bottle
10. Biscuits        : ₹25 per packet
-----------------------------------------------------

Today's Discount Offers:
1. Buy 5 kg or more of Rice and get 5% off on Rice.
2. Buy 3 litres or more of Milk and get 10% off on Milk.
3. Buy 2 packets or more of Biscuits and get 15% off on Biscuits.

Enter item number to buy: 1
Enter quantity of Rice (in kg): 5
Added 5.0 kg of Rice at ₹50 each.

Do you want to add more items? (y/n): y
Enter item number to buy: 5
Enter quantity of Milk (in litre): 6
Added 6.0 litre of Milk at ₹60 each.

Do you want to add more items? (y/n): n

==================== FINAL BILL ====================
5% discount applied on Rice.
Original price: ₹250.00
Rice                5.00 kg     x ₹ 50.00  =  ₹ 237.50
10% discount applied on Milk.
Original price: ₹360.00
Milk                6.00 litre  x ₹ 60.00  =  ₹ 324.00
-------------------------------------------------------
```

```
Subtotal before discount: ₹610.00
GRAND TOTAL: ₹ 561.50
============================================================
        Thank you for shopping at STAR DAILY MART!
                        Visit Again!
Customer Name : XYZ
Phone Number  : 1234567890
```

🏁 Conclusion & Future Enhancements

📋 Conclusion

The "Supermarket Billing System" successfully automates the checkout process for Star Daily Mart. It is robust, handles invalid inputs gracefully, and applies complex discount logic accurately.

🚀 Future Enhancements

- **Database Integration**: Connect to SQL for persistent inventory tracking.
- **GUI Implementation**: Upgrade from Console to Tkinter/PyQt interface.

Designed & Developed by
Chandra Prakash Choudhary

Connect on LinkedIn